CyberTrust KSA Platform: Technical Requirements Specification

Document Version: 1.0
Date: February 20, 2026
Prepared by: Manus AI
Target Audience: Development Team, Technical Architects, DevOps Engineers

---

1. Executive Summary

This document provides comprehensive technical requirements for the CyberTrust KSA platform, a cloud-based AI-driven cybersecurity compliance ecosystem designed to transform NCA (National Cybersecurity Authority) compliance in Saudi Arabia. The platform consolidates 51 distinct functionalities across four categories, requiring a robust, scalable, and secure technical architecture built primarily on **Python 3.11+** and **MySQL 8.0+** with supporting technologies for AI/ML, real-time processing, and multi-cloud integration.

The technical implementation must support the transformation of the traditional 5-day manual audit process into a 2-hour AI-driven experience while maintaining regulatory compliance, data sovereignty, and enterprise-grade security standards.

---

2. Technology Stack Overview

2.1 Backend Technologies

Component	Technology	Version	Justification
Primary Language	Python	3.11+	Extensive AI/ML libraries, rapid development, strong NLP support
Web Framework	FastAPI	0.104+	High performance, async support, automatic API documentation, type safety
Alternative Framework	Django	4.2+ LTS	Admin interface, ORM, built-in security features for complex workflows
Database (Primary)	MySQL	8.0+	ACID compliance, proven scalability, JSON support, strong Saudi market presence
Database (Caching)	Redis	7.2+	In-memory caching, session management, real-time features, pub/sub
Task Queue	Celery	5.3+	Distributed task processing, scheduled jobs, async evidence collection
Message Broker	RabbitMQ / Redis	Latest	Task distribution, event streaming for continuous monitoring
Search Engine	Elasticsearch	8.x	Full-text search for policies, documents, and compliance evidence
Object Storage	MinIO / S3	Latest	Document storage, evidence artifacts, encrypted file management

2.2 AI/ML Technologies

Component	Technology	Purpose
NLP Engine	OpenAI GPT-4 API	Policy analysis, chatbot, document understanding
Local NLP	Hugging Face Transformers	Arabic NLP, offline processing, custom fine-tuning
Arabic Models	CAMeL Tools, AraBERT	Saudi dialect processing, RTL text analysis
ML Framework	scikit-learn, TensorFlow	Risk scoring, predictive analytics, anomaly detection
Document Processing	PyPDF2, pdfplumber, Tesseract OCR	Evidence extraction from PDFs, scanned documents
Computer Vision	OpenCV, Pillow	Diagram analysis, screenshot processing

2.3 Frontend Technologies

Component	Technology	Version	Purpose
Framework	React	18+	Component-based UI, rich ecosystem, mobile support
UI Library	Tailwind CSS	3.x	Rapid styling, RTL support, responsive design
Component Library	shadcn/ui, Ant Design	Latest	Pre-built components, Arabic localization
State Management	Redux Toolkit / Zustand	Latest	Global state, API caching, real-time updates
API Client	Axios, React Query	Latest	HTTP requests, caching, optimistic updates
Charts	Recharts, Chart.js	Latest	Compliance dashboards, risk visualization
Mobile	React Native	0.72+	iOS/Android apps for offline evidence upload

2.4 Infrastructure & DevOps

Component	Technology	Purpose
Cloud Provider	AWS (Primary), Azure (Secondary)	KSA regions (Riyadh, Jeddah), data sovereignty compliance
Containerization	Docker	24.x
Orchestration	Kubernetes (EKS/AKS)	1.28+
CI/CD	GitHub Actions, GitLab CI	Latest
Infrastructure as Code	Terraform	1.6+
Monitoring	Prometheus, Grafana	Latest
Logging	ELK Stack (Elasticsearch, Logstash, Kibana)	8.x
Security Scanning	Trivy, OWASP ZAP	Latest

2.5 Integration Technologies

Integration Type	Technology/Protocol	Purpose
API Standard	RESTful API, OpenAPI 3.0	External integrations, auditor portals
Authentication	OAuth 2.0, JWT	Secure API access, SSO integration
Cloud APIs	AWS SDK (boto3), Azure SDK	Cloud provider integrations (AWS, Azure, AliBaba, STC)
Security Tools	CrowdStrike API, Fortinet API	Automated evidence collection from security tools
HR Systems	SOAP/REST APIs	Employee training records (SAP, Workday)
Ticketing	Jira REST API, ServiceNow API	Change management evidence
Government APIs	Etimad API, Monafasat API	Saudi procurement platform integration
Blockchain	Hyperledger Fabric / Ethereum	Immutable audit trails, smart contracts

---

3. System Architecture

3.1 High-Level Architecture

The platform follows a **microservices architecture** with the following core services:

```
┌──────────────────────────────────────────────────────────────┐
│                    Load Balancer (AWS ALB)                   │
└──────────────────────────────────────────────────────────────┘
                              │
        ┌─────────────────────┼─────────────────────┐
        │                     │                     │
┌───────▼────────┐    ┌───────▼────────┐    ┌──────▼──────┐
│  Web Frontend  │    │   API Gateway  │    │ Mobile Apps │
│   (React)      │    │   (Kong/Nginx) │    │(React Native│
└────────────────┘    └────────────────┘    └─────────────┘
                              │
        ┌─────────────────────┼──────────────────────────┐
        │                     │                          │
┌───────▼────────┐    ┌───────▼────────┐    ┌──────────────▼──┐
│ Auth Service   │    │ Evidence Service│    │  AI/NLP Service  │
│  (FastAPI)     │    │   (FastAPI)     │    │   (FastAPI)      │
└────────────────┘    └────────────────┘    └──────────────────┘
        │                     │                     │
┌───────▼────────┐    ┌───────▼────────┐    ┌──────────▼──────┐
│ Risk Scoring   │    │ Integration Hub │    │ Compliance Engine │
│   Service      │    │   (Celery)      │    │    (Django)       │
└────────────────┘    └────────────────┘    └──────────────────┘
        │                     │                     │
        └─────────────────────┼─────────────────────┘
                              │
        ┌─────────────────────┼─────────────────────┐
        │                     │                     │
┌───────▼────────┐    ┌───────▼────────┐    ┌──────▼──────┐
│  MySQL 8.0+    │    │  Redis Cache   │    │Elasticsearch│
│  (Primary DB)  │    │  (Sessions)    │    │(Search/Logs)│
└────────────────┘    └────────────────┘    └─────────────┘
        │
┌───────▼────────┐
│  S3/MinIO      │
│ (File Storage) │
└────────────────┘
```

3.2 Core Microservices

3.2.1 Authentication & Authorization Service

- **Technology:** FastAPI + JWT + OAuth 2.0

- **Database:** MySQL (users, roles, permissions), Redis (sessions)
- **Features:**
  - Multi-tenant user management
  - Role-Based Access Control (RBAC) with AI anomaly detection (C2)
  - SSO integration with Absher/Muqeem (B8)
  - API key management for integrations
  - Audit logging of all access attempts

3.2.2 AI/NLP Service

- **Technology:** FastAPI + OpenAI API + Hugging Face + CAMeL Tools
- **Features:**
  - Policy document analysis (A1)
  - AI-powered policy generation (D2)
  - Virtual CISO chatbot (A3)
  - Arabic dialect processing (C8)
  - Gap analysis and remediation suggestions (A9)
  - Contract clause analysis (B1)

3.2.3 Evidence Collection Service

- **Technology:** FastAPI + Celery + Cloud SDKs
- **Features:**
  - Automated API integrations with cloud providers (A2)
  - Security tool data extraction (CrowdStrike, Fortinet)
  - HR system integration for training records (C4, D1)
  - Ticketing system integration (Jira, ServiceNow)
  - Scheduled evidence collection jobs (A8)
  - Evidence versioning and repository management

3.2.4 Compliance Engine

- **Technology:** Django + MySQL + Elasticsearch

- **Features:**
  - NCA framework mapping (ECC, TCC, CSCC, PDPL)
  - Multi-framework mapping (D7: ISO 27001, NIST, CIS)
  - Continuous compliance monitoring (B2)
  - Compliance drift detection and alerting
  - Real-time compliance score calculation (D4)
  - Regulatory change tracking (B6)

3.2.5 Risk Scoring Service

- **Technology:** FastAPI + scikit-learn + TensorFlow
- **Features:**
  - Dynamic risk categorization (A4)
  - AI-powered predictive risk analytics (B5)
  - Supply chain risk propagation analysis (B3)
  - Breach likelihood prediction
  - Cyber maturity assessment (D12)
  - Risk quantification for insurance (B10)

3.2.6 Integration Hub

- **Technology:** Celery + RabbitMQ + Redis
- **Features:**
  - Asynchronous task processing
  - API rate limiting and throttling (D13)
  - Third-party API orchestration
  - Webhook management
  - Data transformation and normalization
  - Retry logic and error handling

3.2.7 Auditor Portal Service

- **Technology:** Django + MySQL

- **Features:**
  - Fast-track partnership workflow (A5)
  - Auditor dashboard with pre-mapped evidence
  - One-click approval system
  - Confidence scoring display (A10)
  - Revenue sharing calculation (A12)
  - Audit trail documentation

3.2.8 Reporting & Analytics Service

- **Technology:** FastAPI + Pandas + Plotly
- **Features:**
  - Customizable dashboards (C7)
  - Peer benchmarking analytics (C10)
  - Executive risk communication reports (D10)
  - Compliance ROI calculator (D15)
  - Exportable reports (PDF, Excel)
  - Real-time KPI/KRI tracking

---

4. Database Design (MySQL 8.0+)

4.1 Core Database Schema

4.1.1 User Management Tables

```sql
-- Users table with multi-tenant support
CREATE TABLE users (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    full_name VARCHAR(255) NOT NULL,
    role_id BIGINT NOT NULL,
    is_active BOOLEAN DEFAULT TRUE,
    last_login TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (role_id) REFERENCES roles(id),
    INDEX idx_tenant_email (tenant_id, email),
    INDEX idx_role (role_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Tenants (Organizations/Vendors)
CREATE TABLE tenants (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(255) NOT NULL,
    name_ar VARCHAR(255),
    industry VARCHAR(100),
    risk_category ENUM('general', 'high_risk') DEFAULT 'general',
    parent_tenant_id BIGINT NULL,  -- For multi-entity support (C12)
    subscription_tier ENUM('basic', 'professional', 'enterprise') DEFAULT
'basic',
    compliance_score DECIMAL(5,2) DEFAULT 0.00,
    status ENUM('active', 'suspended', 'pending') DEFAULT 'pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (parent_tenant_id) REFERENCES tenants(id),
    INDEX idx_parent (parent_tenant_id),
    INDEX idx_status (status)
```

```sql
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Roles and Permissions (RBAC)
CREATE TABLE roles (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    permissions JSON,  -- Granular permissions stored as JSON
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Access Logs for AI Auditing (C2)
CREATE TABLE access_logs (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    user_id BIGINT NOT NULL,
    tenant_id BIGINT NOT NULL,
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(100),
    resource_id BIGINT,
    ip_address VARCHAR(45),
    user_agent TEXT,
    anomaly_score DECIMAL(5,2) DEFAULT 0.00,  -- AI anomaly detection
    is_flagged BOOLEAN DEFAULT FALSE,
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id),
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    INDEX idx_user_timestamp (user_id, timestamp),
    INDEX idx_anomaly (is_flagged, anomaly_score)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.2 Compliance Framework Tables

```sql
-- NCA Frameworks and Controls
CREATE TABLE compliance_frameworks (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    code VARCHAR(50) UNIQUE NOT NULL,  -- 'NCA_ECC', 'NCA_TCC', 'PDPL',
'ISO27001'
    name VARCHAR(255) NOT NULL,
    name_ar VARCHAR(255),
    version VARCHAR(50),
    effective_date DATE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE compliance_controls (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    framework_id BIGINT NOT NULL,
    control_id VARCHAR(100) NOT NULL,  -- e.g., 'ECC-1-1'
    title VARCHAR(500) NOT NULL,
    title_ar VARCHAR(500),
    description TEXT,
    description_ar TEXT,
    category VARCHAR(100),
    priority ENUM('critical', 'high', 'medium', 'low') DEFAULT 'medium',
    risk_level ENUM('general', 'high_risk') DEFAULT 'general',
    evidence_requirements JSON,  -- List of required evidence types
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (framework_id) REFERENCES compliance_frameworks(id),
    UNIQUE KEY unique_control (framework_id, control_id),
    INDEX idx_category (category),
    INDEX idx_priority (priority)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Multi-Framework Mapping (D7)
CREATE TABLE framework_mappings (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    source_control_id BIGINT NOT NULL,
    target_control_id BIGINT NOT NULL,
    mapping_strength ENUM('exact', 'partial', 'related') DEFAULT 'partial',
    notes TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (source_control_id) REFERENCES compliance_controls(id),
    FOREIGN KEY (target_control_id) REFERENCES compliance_controls(id),
    INDEX idx_source (source_control_id),
```

```sql
    INDEX idx_target (target_control_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.3 Assessment and Evidence Tables

```sql
-- Compliance Assessments
CREATE TABLE assessments (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    framework_id BIGINT NOT NULL,
    assessment_type ENUM('self', 'auditor', 'continuous') DEFAULT 'self',
    status ENUM('draft', 'in_progress', 'submitted', 'approved', 'rejected')
DEFAULT 'draft',
    risk_score DECIMAL(5,2) DEFAULT 0.00,
    compliance_score DECIMAL(5,2) DEFAULT 0.00,
    started_at TIMESTAMP NULL,
    completed_at TIMESTAMP NULL,
    auditor_id BIGINT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (framework_id) REFERENCES compliance_frameworks(id),
    FOREIGN KEY (auditor_id) REFERENCES users(id),
    INDEX idx_tenant_status (tenant_id, status),
    INDEX idx_auditor (auditor_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Control Compliance Status
CREATE TABLE control_compliance (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    assessment_id BIGINT NOT NULL,
    control_id BIGINT NOT NULL,
    status ENUM('compliant', 'non_compliant', 'partial', 'not_applicable',
'pending') DEFAULT 'pending',
    confidence_score DECIMAL(5,2) DEFAULT 0.00,  -- AI confidence
    gap_description TEXT,
    remediation_plan TEXT,
    evidence_count INT DEFAULT 0,
    last_checked TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (assessment_id) REFERENCES assessments(id),
    FOREIGN KEY (control_id) REFERENCES compliance_controls(id),
    INDEX idx_assessment (assessment_id),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

```sql
-- Evidence Repository
CREATE TABLE evidence (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    assessment_id BIGINT NULL,
    control_id BIGINT NULL,
    evidence_type ENUM('document', 'screenshot', 'api_data', 'certificate',
'log', 'other') DEFAULT 'document',
    file_name VARCHAR(500),
    file_path VARCHAR(1000),  -- S3/MinIO path
    file_hash VARCHAR(128),  -- SHA-256 for integrity
    file_size BIGINT,
    mime_type VARCHAR(100),
    description TEXT,
    ai_analysis_result JSON,  -- NLP analysis results
    is_encrypted BOOLEAN DEFAULT TRUE,
    collection_method ENUM('manual', 'automated', 'api') DEFAULT 'manual',
    collected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    expires_at TIMESTAMP NULL,
    created_by BIGINT NOT NULL,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (assessment_id) REFERENCES assessments(id),
    FOREIGN KEY (control_id) REFERENCES compliance_controls(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
    INDEX idx_tenant_control (tenant_id, control_id),
    INDEX idx_assessment (assessment_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.4 Integration and API Tables

```sql
-- Cloud and Security Tool Integrations (A2)
CREATE TABLE integrations (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    integration_type VARCHAR(100) NOT NULL,  -- 'aws', 'azure',
'crowdstrike', 'jira'
    integration_name VARCHAR(255),
    credentials_encrypted TEXT,  -- Encrypted API keys/tokens
    configuration JSON,
    status ENUM('active', 'inactive', 'error') DEFAULT 'inactive',
    last_sync TIMESTAMP NULL,
    sync_frequency INT DEFAULT 3600,  -- seconds
    error_message TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    INDEX idx_tenant_type (tenant_id, integration_type),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- API Rate Limiting (D13)
CREATE TABLE api_rate_limits (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    api_key_hash VARCHAR(128) NOT NULL,
    endpoint VARCHAR(255),
    request_count INT DEFAULT 0,
    window_start TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    window_duration INT DEFAULT 3600,  -- seconds
    limit_threshold INT DEFAULT 1000,
    is_blocked BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    INDEX idx_tenant_key (tenant_id, api_key_hash),
    INDEX idx_window (window_start)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.5 Continuous Monitoring Tables (B2)

```sql
-- Continuous Compliance Monitoring
CREATE TABLE compliance_monitoring (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    control_id BIGINT NOT NULL,
    monitoring_rule JSON,  -- Rule definition for automated checks
    check_frequency INT DEFAULT 86400,  -- seconds (daily default)
    last_check TIMESTAMP NULL,
    next_check TIMESTAMP NULL,
    current_status ENUM('compliant', 'non_compliant', 'warning', 'unknown')
DEFAULT 'unknown',
    drift_detected BOOLEAN DEFAULT FALSE,
    alert_sent BOOLEAN DEFAULT FALSE,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (control_id) REFERENCES compliance_controls(id),
    INDEX idx_tenant_control (tenant_id, control_id),
    INDEX idx_next_check (next_check)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Compliance Events and Alerts
CREATE TABLE compliance_events (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    event_type VARCHAR(100) NOT NULL,  -- 'drift_detected',
'control_failed', 'evidence_expired'
    severity ENUM('critical', 'high', 'medium', 'low', 'info') DEFAULT
'info',
    control_id BIGINT NULL,
    description TEXT,
    event_data JSON,
    is_resolved BOOLEAN DEFAULT FALSE,
    resolved_at TIMESTAMP NULL,
    resolved_by BIGINT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (control_id) REFERENCES compliance_controls(id),
    FOREIGN KEY (resolved_by) REFERENCES users(id),
    INDEX idx_tenant_severity (tenant_id, severity),
```

```
    INDEX idx_unresolved (is_resolved, created_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.6 Supply Chain and Third-Party Risk Tables (B3)

```sql
-- Vendor/Supplier Management
CREATE TABLE vendors (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,  -- The organization managing this vendor
    vendor_name VARCHAR(255) NOT NULL,
    vendor_type VARCHAR(100),
    risk_level ENUM('low', 'medium', 'high', 'critical') DEFAULT 'medium',
    compliance_score DECIMAL(5,2) DEFAULT 0.00,
    has_nca_cert BOOLEAN DEFAULT FALSE,
    cert_expiry_date DATE NULL,
    data_access_level ENUM('none', 'limited', 'moderate', 'extensive')
DEFAULT 'none',
    geopolitical_risk_score DECIMAL(5,2) DEFAULT 0.00,
    financial_health_score DECIMAL(5,2) DEFAULT 0.00,
    status ENUM('active', 'inactive', 'under_review', 'terminated') DEFAULT
'active',
    onboarded_at TIMESTAMP NULL,
    last_assessment TIMESTAMP NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    INDEX idx_tenant_risk (tenant_id, risk_level),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Vendor Relationships (4th/5th party mapping)
CREATE TABLE vendor_relationships (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    parent_vendor_id BIGINT NOT NULL,
    child_vendor_id BIGINT NOT NULL,
    relationship_type ENUM('subcontractor', 'supplier', 'partner',
'service_provider') DEFAULT 'supplier',
    data_flow_description TEXT,
    risk_propagation_score DECIMAL(5,2) DEFAULT 0.00,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (parent_vendor_id) REFERENCES vendors(id),
    FOREIGN KEY (child_vendor_id) REFERENCES vendors(id),
    INDEX idx_parent (parent_vendor_id),
    INDEX idx_child (child_vendor_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.7 Training and Certification Tables (D1)

```sql
-- Employee Training Management
CREATE TABLE training_courses (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    course_code VARCHAR(100) UNIQUE NOT NULL,
    title VARCHAR(500) NOT NULL,
    title_ar VARCHAR(500),
    description TEXT,
    course_type ENUM('awareness', 'technical', 'compliance', 'leadership')
DEFAULT 'awareness',
    duration_minutes INT,
    passing_score INT DEFAULT 80,
    related_controls JSON,  -- NCA controls addressed by this training
    is_mandatory BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

CREATE TABLE employee_training (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    employee_id VARCHAR(255) NOT NULL,  -- From HR system
    employee_name VARCHAR(255),
    course_id BIGINT NOT NULL,
    enrollment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    completion_date TIMESTAMP NULL,
    score INT NULL,
    status ENUM('enrolled', 'in_progress', 'completed', 'failed', 'expired')
DEFAULT 'enrolled',
    certificate_issued BOOLEAN DEFAULT FALSE,
    certificate_path VARCHAR(1000),
    expiry_date DATE NULL,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (course_id) REFERENCES training_courses(id),
    INDEX idx_tenant_employee (tenant_id, employee_id),
    INDEX idx_status (status),
    INDEX idx_expiry (expiry_date)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.8 Policy Management Tables (D2)

```sql
-- AI-Generated Policies
CREATE TABLE policies (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    policy_type VARCHAR(100) NOT NULL,  -- 'information_security',
'data_classification', 'incident_response'
    title VARCHAR(500) NOT NULL,
    title_ar VARCHAR(500),
    content TEXT NOT NULL,  -- Full policy text
    content_ar TEXT,
    version VARCHAR(50) DEFAULT '1.0',
    generation_method ENUM('ai_generated', 'template', 'manual', 'imported')
DEFAULT 'manual',
    ai_confidence_score DECIMAL(5,2) DEFAULT 0.00,
    status ENUM('draft', 'review', 'approved', 'archived') DEFAULT 'draft',
    approved_by BIGINT NULL,
    approved_at TIMESTAMP NULL,
    effective_date DATE NULL,
    review_date DATE NULL,
    related_controls JSON,  -- NCA controls addressed
    created_by BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
    FOREIGN KEY (approved_by) REFERENCES users(id),
    INDEX idx_tenant_type (tenant_id, policy_type),
    INDEX idx_status (status)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

-- Policy Version History
CREATE TABLE policy_versions (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    policy_id BIGINT NOT NULL,
    version VARCHAR(50) NOT NULL,
    content TEXT NOT NULL,
    change_summary TEXT,
    created_by BIGINT NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (policy_id) REFERENCES policies(id),
    FOREIGN KEY (created_by) REFERENCES users(id),
```

```
    INDEX idx_policy (policy_id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.9 Incident Response Tables (B7)

```
-- Security Incidents
CREATE TABLE security_incidents (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    incident_id VARCHAR(100) UNIQUE NOT NULL,  -- e.g., 'INC-2026-001'
    title VARCHAR(500) NOT NULL,
    description TEXT,
    severity ENUM('critical', 'high', 'medium', 'low') DEFAULT 'medium',
    incident_type VARCHAR(100),  -- 'data_breach', 'ransomware', 'phishing'
    status ENUM('detected', 'investigating', 'contained', 'resolved',
'closed') DEFAULT 'detected',
    detected_at TIMESTAMP NOT NULL,
    contained_at TIMESTAMP NULL,
    resolved_at TIMESTAMP NULL,
    nca_notification_required BOOLEAN DEFAULT FALSE,
    nca_notified_at TIMESTAMP NULL,
    response_plan JSON,  -- Playbook steps and execution
    evidence_preserved BOOLEAN DEFAULT FALSE,
    assigned_to BIGINT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    FOREIGN KEY (assigned_to) REFERENCES users(id),
    INDEX idx_tenant_status (tenant_id, status),
    INDEX idx_severity (severity),
    INDEX idx_detected (detected_at)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.1.10 Blockchain Audit Trail Tables (B9)

```sql
-- Blockchain Verification Records
CREATE TABLE blockchain_records (
    id BIGINT PRIMARY KEY AUTO_INCREMENT,
    tenant_id BIGINT NOT NULL,
    record_type VARCHAR(100) NOT NULL,  -- 'evidence_upload',
'assessment_approval', 'policy_approval'
    record_id BIGINT NOT NULL,  -- ID of the related record
    blockchain_hash VARCHAR(128) NOT NULL,  -- Transaction hash on
blockchain
    blockchain_network VARCHAR(100),  -- 'hyperledger', 'ethereum'
    block_number BIGINT,
    timestamp_utc TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    verification_url VARCHAR(1000),  -- Public verification URL
    is_verified BOOLEAN DEFAULT FALSE,
    FOREIGN KEY (tenant_id) REFERENCES tenants(id),
    INDEX idx_tenant_type (tenant_id, record_type),
    INDEX idx_hash (blockchain_hash)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

4.2 Database Performance Optimization

4.2.1 Indexing Strategy

- **Primary Keys:** All tables use `BIGINT AUTO_INCREMENT` for scalability
- **Foreign Keys:** Indexed automatically for join performance
- **Composite Indexes:** Created for common query patterns (tenant_id + status, tenant_id + control_id)
- **Full-Text Indexes:** On policy content, descriptions for search functionality

4.2.2 Partitioning

```sql
-- Partition large tables by date for performance
ALTER TABLE access_logs
PARTITION BY RANGE (YEAR(timestamp)) (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p2026 VALUES LESS THAN (2027),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);

ALTER TABLE compliance_events
PARTITION BY RANGE (YEAR(created_at)) (
    PARTITION p2024 VALUES LESS THAN (2025),
    PARTITION p2025 VALUES LESS THAN (2026),
    PARTITION p2026 VALUES LESS THAN (2027),
    PARTITION p_future VALUES LESS THAN MAXVALUE
);
```

4.2.3 Caching Strategy (Redis)

- **Session Data:** User sessions, JWT tokens (TTL: 24 hours)
- **Compliance Scores:** Cached for 1 hour, invalidated on evidence update
- **API Responses:** Common queries cached for 5 minutes
- **Rate Limiting:** Request counters with sliding window
- **Real-time Data:** Pub/sub for compliance alerts and notifications

---

5. API Design

5.1 RESTful API Endpoints

5.1.1 Authentication Endpoints

```
POST   /api/v1/auth/register          - Register new tenant/user
POST   /api/v1/auth/login             - Login and get JWT token
POST   /api/v1/auth/logout            - Invalidate session
POST   /api/v1/auth/refresh           - Refresh JWT token
GET    /api/v1/auth/me                - Get current user profile
```

5.1.2 Assessment Endpoints

```
POST   /api/v1/assessments            - Create new assessment
GET    /api/v1/assessments            - List assessments (with filters)
GET    /api/v1/assessments/{id}       - Get assessment details
PUT    /api/v1/assessments/{id}       - Update assessment
DELETE /api/v1/assessments/{id}       - Delete assessment
POST   /api/v1/assessments/{id}/submit - Submit for auditor review
GET    /api/v1/assessments/{id}/score  - Get compliance score
```

5.1.3 Evidence Endpoints

```
POST   /api/v1/evidence               - Upload evidence
GET    /api/v1/evidence               - List evidence (with filters)
GET    /api/v1/evidence/{id}          - Get evidence details
DELETE /api/v1/evidence/{id}          - Delete evidence
POST   /api/v1/evidence/analyze       - Trigger AI analysis
GET    /api/v1/evidence/{id}/download - Download evidence file
```

5.1.4 Compliance Endpoints

```
GET    /api/v1/compliance/frameworks  - List available frameworks
GET    /api/v1/compliance/controls    - List controls (with filters)
GET    /api/v1/compliance/score       - Get real-time compliance score (D4)
GET    /api/v1/compliance/gaps        - Get compliance gaps
POST   /api/v1/compliance/remediate   - Get remediation suggestions
```

5.1.5 Integration Endpoints

```
POST   /api/v1/integrations           - Add new integration
GET    /api/v1/integrations           - List integrations
PUT    /api/v1/integrations/{id}      - Update integration
DELETE /api/v1/integrations/{id}      - Remove integration
POST   /api/v1/integrations/{id}/sync - Trigger manual sync
GET    /api/v1/integrations/{id}/status - Get sync status
```

5.1.6 AI/NLP Endpoints

```
POST   /api/v1/ai/analyze-policy      - Analyze policy document (A1)
POST   /api/v1/ai/generate-policy     - Generate policy from template (D2)
POST   /api/v1/ai/chatbot             - Virtual CISO chat (A3)
POST   /api/v1/ai/gap-analysis        - Perform gap analysis (A9)
POST   /api/v1/ai/remediation         - Get remediation suggestions
```

5.1.7 Vendor Management Endpoints (B3)

```
POST   /api/v1/vendors                - Add vendor
GET    /api/v1/vendors                - List vendors
GET    /api/v1/vendors/{id}           - Get vendor details
PUT    /api/v1/vendors/{id}           - Update vendor
GET    /api/v1/vendors/{id}/risk      - Get vendor risk assessment
GET    /api/v1/vendors/supply-chain   - Get supply chain map
```

5.2 API Security

- **Authentication:** JWT tokens with 24-hour expiry
- **Authorization:** RBAC with granular permissions
- **Rate Limiting:** 1000 requests/hour per tenant (D13)
- **Input Validation:** Pydantic models for all request bodies
- **Output Sanitization:** Prevent data leakage across tenants
- **Encryption:** TLS 1.3 for all API traffic
- **API Keys:** For external integrations (government APIs, auditor portals)

5.3 API Documentation

- **OpenAPI 3.0 Specification:** Auto-generated from FastAPI
- **Interactive Docs:** Swagger UI at `/docs`, ReDoc at `/redoc`
- **Versioning:** URL-based versioning (`/api/v1/` , `/api/v2/`)
- **Changelog:** Maintained for breaking changes

---

6. AI/ML Implementation

6.1 Natural Language Processing (NLP)

6.1.1 Policy Analysis (A1)

```
# Technology: OpenAI GPT-4 API + Hugging Face
# Input: PDF policy documents
# Output: Compliance gap analysis, missing clauses

from openai import OpenAI
from transformers import pipeline

# Arabic NLP for Saudi dialect
arabic_nlp = pipeline("text-classification", model="aubmindlab/bert-base-arabertv2")

# Policy analysis function
def analyze_policy(policy_text, framework="NCA_ECC"):
    # Extract key sections using NLP
    # Cross-reference with NCA controls
    # Identify missing clauses
    # Return gap analysis with confidence scores
    pass
```

6.1.2 AI Policy Generation (D2)

```
# Technology: OpenAI GPT-4 API + Templates
# Input: Organization profile, industry, risk level
# Output: Customized policy document

def generate_policy(tenant_id, policy_type, customization_params):
    # Load policy template
    # Customize based on tenant profile
    # Generate using GPT-4
    # Validate against NCA requirements
    # Return policy with version history
    pass
```

6.1.3 Virtual CISO Chatbot (A3)

```
# Technology: OpenAI GPT-4 API + RAG (Retrieval-Augmented Generation)
# Input: User question in Arabic/English
# Output: Contextual compliance guidance

from langchain import OpenAI, VectorStore

# Knowledge base: NCA frameworks, PDPL, best practices
knowledge_base = VectorStore.from_documents(nca_docs)

def chatbot_response(user_message, tenant_context):
    # Retrieve relevant NCA controls
    # Generate response using GPT-4
    # Provide templates if applicable
    # Track conversation history
    pass
```

6.2 Machine Learning Models

6.2.1 Risk Scoring (A4, B5)

```
# Technology: scikit-learn Random Forest
# Input: Tenant profile, industry, questionnaire responses
# Output: Risk category, compliance score prediction

from sklearn.ensemble import RandomForestClassifier

# Train model on historical assessment data
risk_model = RandomForestClassifier(n_estimators=100)
risk_model.fit(X_train, y_train)

def calculate_risk_score(tenant_data):
    # Feature engineering
    # Predict risk category
    # Calculate compliance score
    # Return with confidence interval
    pass
```

6.2.2 Anomaly Detection (C2)

```python
# Technology: Isolation Forest for access log anomalies
# Input: User access patterns
# Output: Anomaly score, flagged suspicious activity

from sklearn.ensemble import IsolationForest

anomaly_detector = IsolationForest(contamination=0.01)

def detect_access_anomaly(user_id, access_log):
    # Extract features: time, location, resource accessed
    # Calculate anomaly score
    # Flag if score > threshold
    # Alert security team
    pass
```

6.2.3 Predictive Compliance Forecasting (D5)

```python
# Technology: LSTM (Long Short-Term Memory) neural network
# Input: Historical compliance data, evidence collection patterns
# Output: Predicted compliance score, audit failure risk

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

def forecast_compliance(tenant_id, forecast_horizon_days=90):
    # Load historical compliance data
    # Train LSTM model
    # Predict future compliance score
    # Identify high-risk controls
    # Generate alerts for expiring evidence
    pass
```

6.3 AI Model Training and Deployment

- **Training Data:** Anonymized historical assessments, NCA frameworks, policy documents
- **Model Registry:** MLflow for version control and experiment tracking

- **Deployment:** TensorFlow Serving or FastAPI endpoints
- **Monitoring:** Model performance metrics, drift detection
- **Retraining:** Quarterly retraining with new data (C13)

---

7. Security Requirements

7.1 Data Encryption

7.1.1 End-to-End Encryption (C1)

- **Client-Side Encryption:** AES-256 encryption before upload
- **Zero-Knowledge Architecture:** Platform cannot decrypt without user key
- **Key Management:** AWS KMS or HashiCorp Vault
- **Evidence Storage:** Encrypted at rest in S3/MinIO

7.1.2 Data in Transit

- **TLS 1.3:** All API communications
- **Certificate Pinning:** Mobile apps
- **VPN Integration:** For on-premise integrations

7.1.3 Data at Rest

- **MySQL Encryption:** InnoDB tablespace encryption
- **Redis Encryption:** RDB/AOF file encryption
- **Backup Encryption:** Automated encrypted backups to S3

7.2 Data Residency (C3)

- **KSA Regions Only:** AWS Riyadh, Azure Riyadh/Jeddah
- **Automated Verification:** Cloud region checker runs hourly
- **Alerts:** Immediate notification if data detected outside KSA
- **Compliance Reports:** Monthly data residency compliance reports

7.3 Access Control

- **Multi-Factor Authentication (MFA):** Required for all users
- **RBAC:** Granular role-based permissions (C2)
- **API Key Rotation:** Automated 90-day rotation
- **Session Management:** Redis-based with 24-hour expiry
- **IP Whitelisting:** Optional for enterprise tenants

7.4 Audit Logging

- **Comprehensive Logging:** All user actions, API calls, data access
- **Immutable Logs:** Write-once storage in Elasticsearch
- **Blockchain Verification:** Critical actions recorded on blockchain (B9)
- **Retention:** 7 years for compliance requirements
- **SIEM Integration:** Export to customer SIEM systems

---

8. Performance Requirements

8.1 Response Time

Operation	Target Response Time	Maximum
API Endpoints (GET)	< 200ms	500ms
API Endpoints (POST/PUT)	< 500ms	1s
AI Policy Analysis	< 30s	60s
Evidence Upload (10MB)	< 5s	10s
Dashboard Load	< 1s	2s
Chatbot Response	< 3s	5s

8.2 Scalability

- **Concurrent Users:** Support 10,000+ concurrent users
- **Tenants:** Support 50,000+ tenants
- **Assessments:** 100,000+ active assessments
- **Evidence Storage:** Petabyte-scale with S3
- **API Throughput:** 10,000 requests/second (with rate limiting)

8.3 Availability

- **Uptime SLA:** 99.9% (8.76 hours downtime/year)
- **High Availability:** Multi-AZ deployment in AWS/Azure
- **Disaster Recovery:** RTO < 4 hours, RPO < 1 hour
- **Backup Frequency:** Hourly incremental, daily full backups
- **Failover:** Automated failover for database and services

---

9. Integration Requirements

9.1 Cloud Provider Integrations (A2)

Provider	API/SDK	Authentication	Data Collected
AWS	boto3 (Python SDK)	IAM Role / Access Keys	Security groups, encryption status, MFA, CloudTrail logs
Azure	Azure SDK for Python	Service Principal	Network security groups, Key Vault, Azure AD MFA
AliBaba Cloud	AliBaba Cloud SDK	Access Key	Security groups, encryption, RAM policies
STC Cloud	REST API	API Key	Network config, security settings

9.2 Security Tool Integrations

Tool	API	Data Collected
CrowdStrike	Falcon API	Endpoint protection status, threat detections
Fortinet	FortiGate API	Firewall rules, IPS/IDS events, VPN status
Nessus/Qualys	REST API	Vulnerability scan results, patch status (D8)
Microsoft Defender	Graph API	Endpoint compliance, threat intelligence

9.3 HR and Operational Integrations (C4)

System	Integration Method	Data Collected
SAP SuccessFactors	SOAP/REST API	Employee training records, certifications
Workday	REST API	Employee data, training completion
Jira	REST API	Change management tickets, issue tracking
ServiceNow	REST API	Incident tickets, change requests
Microsoft Intune	Graph API	Device compliance, MDM status

9.4 Government Platform Integration (D6)

Platform	Integration	Purpose
Etimad	REST API (if available)	Vendor registration sync, tender requirements
Monafasat	REST API (if available)	Procurement compliance requirements
Absher/Muqeem	OAuth 2.0	User authentication for Saudi nationals/residents

---

10. Deployment Architecture

10.1 Kubernetes Deployment

```yaml
# Example Kubernetes deployment for FastAPI service
apiVersion: apps/v1
kind: Deployment
metadata:
  name: cybertrust-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: cybertrust-api
  template:
    metadata:
      labels:
        app: cybertrust-api
    spec:
      containers:
      - name: api
        image: cybertrust/api:latest
        ports:
        - containerPort: 8000
        env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: url
        - name: REDIS_URL
          valueFrom:
            secretKeyRef:
              name: redis-credentials
              key: url
        resources:
          requests:
            memory: "512Mi"
            cpu: "500m"
          limits:
            memory: "1Gi"
            cpu: "1000m"
        livenessProbe:
          httpGet:
```

```yaml
            path: /health
            port: 8000
          initialDelaySeconds: 30
          periodSeconds: 10
        readinessProbe:
          httpGet:
            path: /ready
            port: 8000
          initialDelaySeconds: 10
          periodSeconds: 5
```

10.2 Infrastructure as Code (Terraform)

```
# Example Terraform for AWS RDS MySQL
resource "aws_db_instance" "cybertrust_mysql" {
  identifier           = "cybertrust-mysql-prod"
  engine               = "mysql"
  engine_version       = "8.0.35"
  instance_class       = "db.r6g.2xlarge"
  allocated_storage    = 1000
  storage_type         = "gp3"
  storage_encrypted    = true
  kms_key_id           = aws_kms_key.rds.arn

  db_name              = "cybertrust"
  username             = var.db_username
  password             = var.db_password

  multi_az             = true
  availability_zone    = "me-south-1a"

  backup_retention_period = 30
  backup_window        = "03:00-04:00"
  maintenance_window   = "sun:04:00-sun:05:00"

  enabled_cloudwatch_logs_exports = ["error", "general", "slowquery"]

  vpc_security_group_ids = [aws_security_group.rds.id]
  db_subnet_group_name   = aws_db_subnet_group.main.name

  tags = {
    Name        = "CyberTrust MySQL Production"
    Environment = "production"
    Compliance  = "NCA-PDPL"
  }
}
```

10.3 CI/CD Pipeline (GitHub Actions)

```yaml
name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest pytest-cov
      - name: Run tests
        run: pytest --cov=app tests/
      - name: Security scan
        run: |
          pip install bandit safety
          bandit -r app/
          safety check

  build:
    needs: test
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Build Docker image
        run: docker build -t cybertrust/api:${{ github.sha }} .
      - name: Scan image
        run: |
          docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
            aquasec/trivy image cybertrust/api:${{ github.sha }}
      - name: Push to registry
        run: docker push cybertrust/api:${{ github.sha }}
```

```yaml
  deploy:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy to Kubernetes
        run: |
          kubectl set image deployment/cybertrust-api \
            api=cybertrust/api:${{ github.sha }}
          kubectl rollout status deployment/cybertrust-api
```

---

11. Development Environment Setup

11.1 Local Development (Docker Compose)

```yaml
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root_password
      MYSQL_DATABASE: cybertrust_dev
      MYSQL_USER: cybertrust
      MYSQL_PASSWORD: dev_password
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  redis:
    image: redis:7.2-alpine
    ports:
      - "6379:6379"

  elasticsearch:
    image: elasticsearch:8.11.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false
    ports:
      - "9200:9200"

  api:
    build: .
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
    volumes:
      - .:/app
    ports:
      - "8000:8000"
    environment:
      - 
DATABASE_URL=mysql+pymysql://cybertrust:dev_password@mysql:3306/cybertrust_dev
      - REDIS_URL=redis://redis:6379/0
      - OPENAI_API_KEY=${OPENAI_API_KEY}
```

```yaml
    depends_on:
      - mysql
      - redis

  celery_worker:
    build: .
    command: celery -A app.celery_app worker --loglevel=info
    volumes:
      - .:/app
    environment:
      - 
DATABASE_URL=mysql+pymysql://cybertrust:dev_password@mysql:3306/cybertrust_dev
      - REDIS_URL=redis://redis:6379/0
    depends_on:
      - mysql
      - redis

volumes:
  mysql_data:
```

11.2 Python Dependencies (requirements.txt)

```
# Web Framework
fastapi==0.104.1
uvicorn[standard]==0.24.0
pydantic==2.5.0
pydantic-settings==2.1.0

# Database
sqlalchemy==2.0.23
pymysql==1.1.0
alembic==1.13.0
redis==5.0.1

# Task Queue
celery==5.3.4
flower==2.0.1

# AI/ML
openai==1.3.7
transformers==4.35.2
torch==2.1.1
scikit-learn==1.3.2
tensorflow==2.15.0
camel-tools==1.5.2

# NLP
spacy==3.7.2
nltk==3.8.1
pdfplumber==0.10.3
pypdf2==3.0.1
pytesseract==0.3.10

# Cloud SDKs
boto3==1.34.0
azure-identity==1.15.0
azure-storage-blob==12.19.0

# Security
cryptography==41.0.7
pyjwt==2.8.0
passlib[bcrypt]==1.7.4
python-multipart==0.0.6

# Monitoring
```

```
prometheus-client==0.19.0
sentry-sdk==1.39.1

# Testing
pytest==7.4.3
pytest-asyncio==0.21.1
pytest-cov==4.1.0
httpx==0.25.2

# Utilities
python-dotenv==1.0.0
httpx==0.25.2
aiofiles==23.2.1
pillow==10.1.0
```

---

12. Testing Requirements

12.1 Unit Testing

- **Framework:** pytest
- **Coverage Target:** > 80%
- **Mocking:** pytest-mock for external APIs
- **Fixtures:** Database fixtures for reproducible tests

12.2 Integration Testing

- **API Testing:** httpx for FastAPI endpoint testing
- **Database Testing:** Test database with sample data
- **External API Mocking:** responses library for cloud/security tool APIs

12.3 Performance Testing

- **Load Testing:** Locust for API load testing
- **Target:** 1000 concurrent users, 10,000 requests/minute
- **Database Performance:** Query optimization with EXPLAIN ANALYZE

12.4 Security Testing

- **SAST:** Bandit for Python code scanning
- **DAST:** OWASP ZAP for API security testing
- **Dependency Scanning:** Safety for vulnerable packages
- **Container Scanning:** Trivy for Docker image vulnerabilities

---

13. Monitoring and Observability

13.1 Application Monitoring

- **Metrics:** Prometheus for custom metrics (API latency, queue length, compliance scores)
- **Dashboards:** Grafana for visualization
- **Alerting:** PagerDuty integration for critical alerts
- **APM:** Sentry for error tracking and performance monitoring

13.2 Infrastructure Monitoring

- **Cloud Metrics:** AWS CloudWatch, Azure Monitor
- **Kubernetes:** Prometheus Operator, kube-state-metrics
- **Database:** MySQL slow query log, performance schema
- **Redis:** Redis INFO metrics

13.3 Log Aggregation

- **ELK Stack:** Elasticsearch, Logstash, Kibana
- **Log Levels:** DEBUG (dev), INFO (staging), WARNING/ERROR (production)
- **Structured Logging:** JSON format for machine parsing
- **Retention:** 90 days hot storage, 7 years cold storage (compliance)

---

14. Documentation Requirements

14.1 API Documentation

- **OpenAPI Specification:** Auto-generated from FastAPI
- **Interactive Docs:** Swagger UI, ReDoc
- **Code Examples:** Python, JavaScript, cURL
- **Postman Collection:** For manual testing

14.2 Developer Documentation

- **README:** Setup instructions, architecture overview
- **Architecture Decision Records (ADRs):** Document key technical decisions
- **Database Schema Diagrams:** ER diagrams for all tables
- **Deployment Guides:** Step-by-step for AWS, Azure, on-premise

14.3 Operational Documentation

- **Runbooks:** Incident response procedures
- **Monitoring Playbooks:** Alert response guidelines
- **Backup/Restore Procedures:** Database recovery steps
- **Scaling Guidelines:** When and how to scale services

---

15. Compliance and Regulatory Requirements

15.1 NCA Compliance

- **Data Encryption:** AES-256 for data at rest and in transit
- **Access Control:** Multi-factor authentication, RBAC
- **Audit Logging:** Comprehensive logging of all security events
- **Incident Response:** 72-hour notification mechanism (B7)

- **Data Residency:** KSA regions only (C3)

15.2 PDPL Compliance

- **Data Classification:** Automated sensitive data identification
- **Consent Management:** User consent tracking (B4)
- **Data Subject Rights:** Automated workflows for access, deletion requests
- **Cross-Border Transfers:** Compliance checker for international data flows
- **Privacy Impact Assessments:** Automated PIA generation

15.3 ISO 27001 Alignment

- **Information Security Management System (ISMS):** Policy framework
- **Risk Assessment:** Continuous risk monitoring
- **Asset Management:** Inventory of all data assets
- **Access Control:** Principle of least privilege

---

16. Appendices

16.1 Glossary

Term	Definition
NCA	National Cybersecurity Authority (Saudi Arabia)
ECC	Essential Cybersecurity Controls
TCC	Third-Party Cybersecurity Controls
CSCC	Cloud Security Controls
PDPL	Personal Data Protection Law (Saudi Arabia)
RBAC	Role-Based Access Control
JWT	JSON Web Token
API	Application Programming Interface
NLP	Natural Language Processing
ML	Machine Learning
SIEM	Security Information and Event Management

16.2 References

1. National Cybersecurity Authority (NCA) - Essential Cybersecurity Controls (ECC)
2. Saudi Personal Data Protection Law (PDPL) - 2021
3. ISO/IEC 27001:2022 - Information Security Management
4. NIST Cybersecurity Framework v1.1
5. CIS Controls v8
6. FastAPI Documentation - https://fastapi.tiangolo.com/
7. MySQL 8.0 Reference Manual - https://dev.mysql.com/doc/
8. OpenAI API Documentation - https://platform.openai.com/docs/

16.3 Version History

Version	Date	Author	Changes
1.0	2026-02-20	Manus AI	Initial technical requirements specification

---

Document Approval:

This technical requirements document serves as the authoritative specification for the CyberTrust KSA platform development. All development teams must adhere to the technologies, architectures, and standards defined herein.

Next Steps:

1. Review and approval by technical leadership
2. Development environment setup
3. Database schema implementation
4. API development (Phase 1 endpoints)
5. AI/ML model training and integration
6. Security hardening and penetration testing
7. Performance testing and optimization
8. Production deployment to KSA regions

End of Technical Requirements Document